

HEAP Memory

- store objects
- There is no order of allocating the memory
- Garbage collector is used to delete the unreferenced object from the heap.

■ Mark and Sweep Algorithm

■ Types of GC

□ Single GC

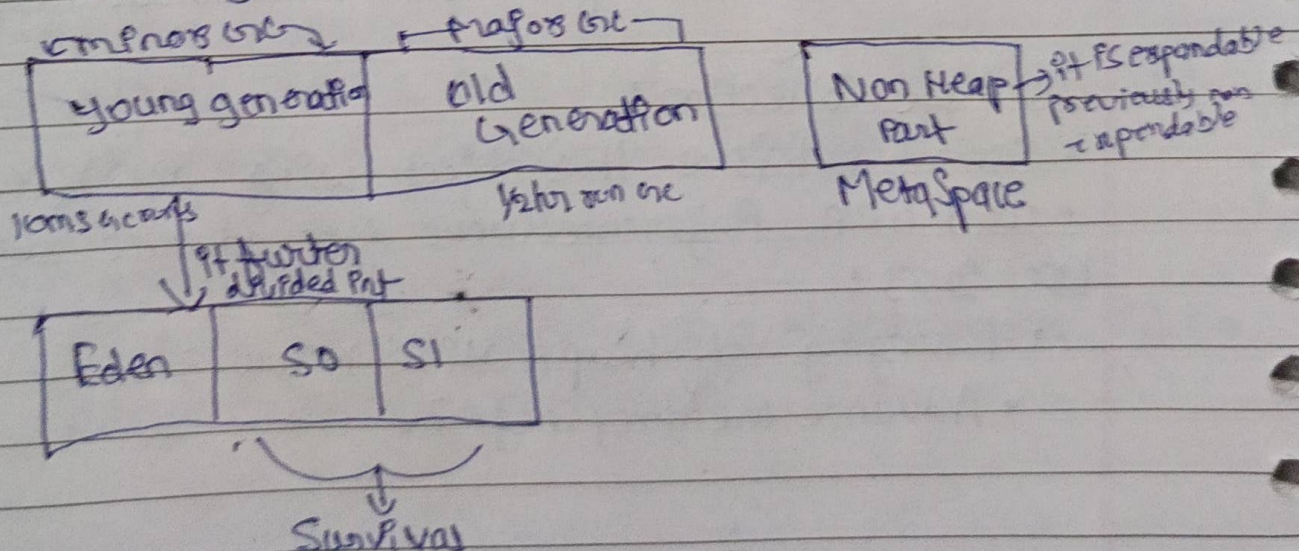
□ Parallel GC → default version of GC in Java 8
2 GCs, 4 GCs

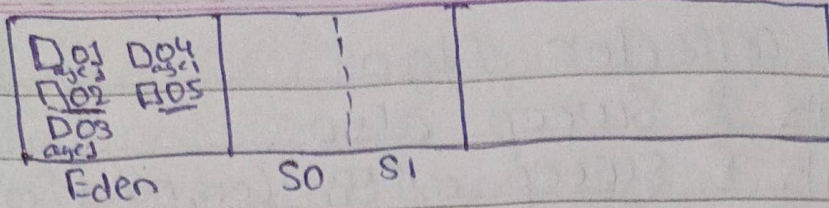
□ CMS (Concurrent Mark Sweep)

□ G1 Garbage collector

- Heap memory is shared with all the threads.
- Heap also contains the string pool
- When heap memory goes full, it throws "java.lang.OutOfMemoryError"
- Heap memory is further divided into:
 - Young Generation (minor GC happens here)
 - Eden
 - Survivor
 - Old/Tenured Generation (major GC happens here)

→ Internally How Heap memory looks

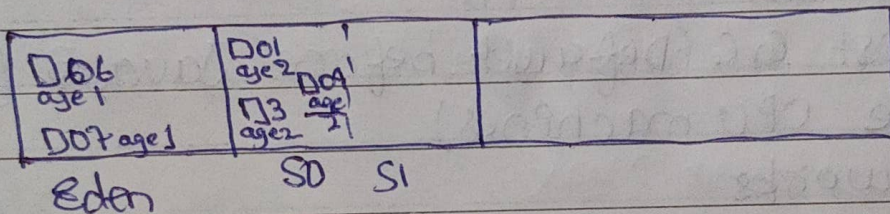




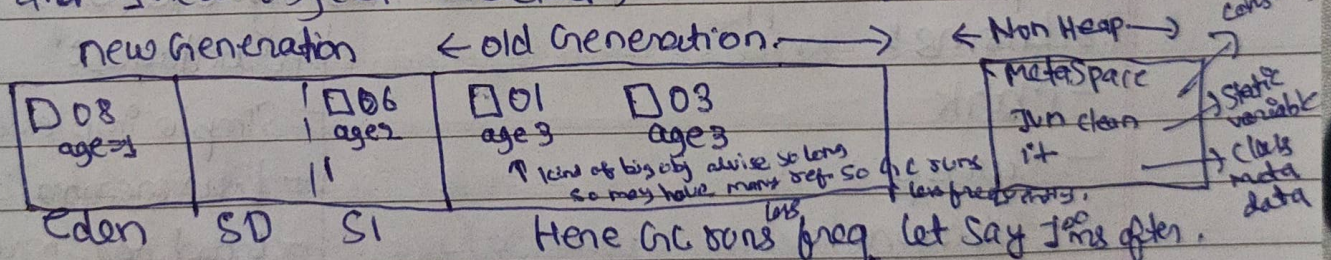
let threshold freq
= 3

let say 5 obj created and its 1st enter into Eden
after every gc cycle run a process runs called
mark & Sweep from eden to Survival block
and each time (mark Survival its age incremented
by 1) and there is fixed one threshold freq if
any object still alive and reached upto threshold
freq it is transfered to old Generation block
where gc cycles runs very frequently

At 10ms GC runs at that time O2 = null, O5 = null
and 2 new object created O6, O7



Again at next 10ms GC runs and let say O4 lost reference
and 1 new object created, O7 lost reference



Sweep algo do 2 thing remove unreferenced obj and move referenced to Survival
old generation Heap memory

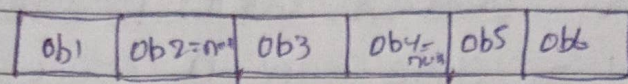
What is metaspace?

→ Before Java 8 metaspace comes under Heap memory Part
as known as Permgen due to this cause memory overflow
very frequently.

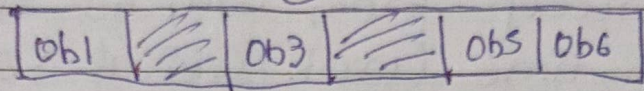
But in the update of Java 8 Permgen is changed to metaspace
and metaspace is related to class variable i.e. All constants, static
variables stored in it

Garbage Collector Algo

- ↳ Mark & Sweep also
- ↳ Mark & Sweep with Compaction



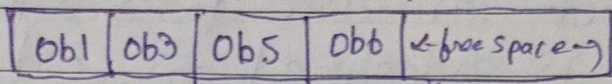
↓



Small holes created
→ that's why despite of having free space program not able to load in memory.

↓

compaction happen all freed space moved right.



if GC app^{work} very slow
↳ that can be used further
very slow, GC very expensive
all threads, process become stop when GC is in running, we say world stop when GC runs
Application become Pause

version of GC

① Serial GC

→ JDK : oldest GC (Default before Java 5 for single CPU machines)

How it works

- uses one thread for garbage collection
- stops all applⁿ threads using GC (Stop-the-world)
- Performs
 - Mark
 - Sweep
 - compact

Advantages:

- Simple implementation
- Low memory overhead
- Good for small application

Application Running

↓

Stop all Application threads

↓

Garbage Collection

↓

Resume Application

Disadvantages

- Long pauses times
- Doesn't utilize multiple CPU cores

Best For:

- Small desktop applⁿ
- Single core machines

2. Parallel GC (Throughput GC)

Introduced : Java 5

Why was it introduced ?

→ Modern Computers Started having multiple CPU cores
Instead of using one thread,

Parallel GC :

CPU1 GC

CPU2 GC

CPU3 GC

CPU4 GC

} Multiple thread collect garbage
Simultaneously.

Characteristics :

- Multiple GC threads
- Still Stop the world
- Much faster than Serial GC
- Maximizes throughput

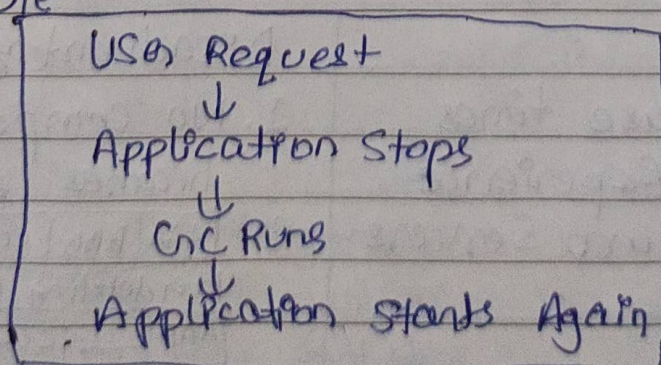
Advantages

- Faster GC
- Better CPU utilization
- Good for batch applications

Disadvantages :

Application still pauses completely.

Example



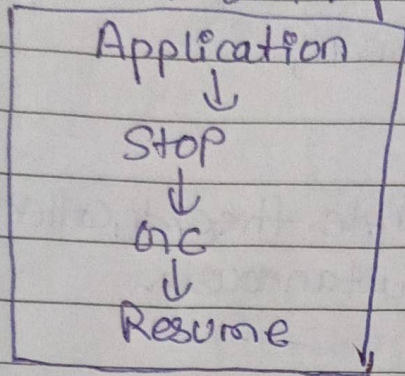
Large heaps can still produce long pauses

Page No. :
Date : / /

3. CMS (Concurrent Mark Sweep) Introduced: Jan 95

Why was CMS introduced?

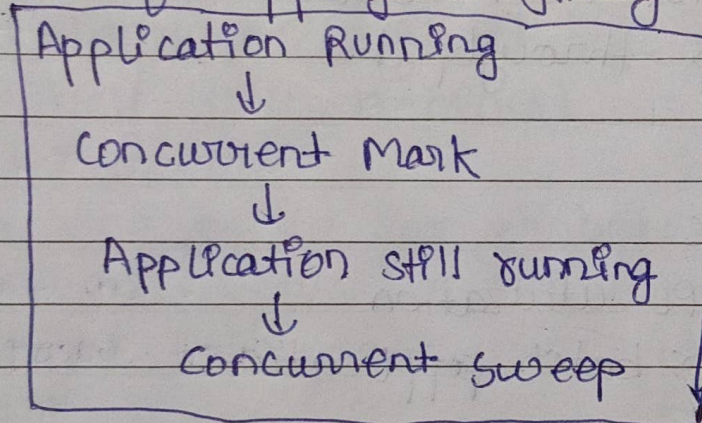
→ Problem with parallel GC:



Users noticed noticeable pauses
Goal: Reduce application pause time.

How CMS works

→ Instead of stopping everything



GC runs alongside the application, Hence the name concurrent Mark Sweep.

Advantages:

- Very low pause times
- Better user experience
- Great for web servers.

Disadvantages

1. No compaction - Memory becomes fragmented

Ex:

AAA	BBB	ccc	ddd
-----	-----	-----	-----

after deleting BBB

AAA	///	ccc	ddd
-----	-----	-----	-----

Eventually there isn't enough contiguous space.

Disadvantages of CMS

- ② More CPU usage → GC and application compete for CPU.
- ③ Concurrent mode failure → If fragmentation become severe.

Because of these problems CMS was removed.
Xend X

Type ④ G1 GC (Garbage First) Introduced : Java 7
Now default in Java 9

Why G1 ?

→ CMS had

- fragmentation
- Full GC
- Poor large heap performance

Need :

- Low pauses
- Large heaps
- No fragmentation

How G1 works

→ Instead of dividing memory into young and old only
Heap is divided into many small regions.

R1	R2	R3
R4	R5	R6
R7	R8	R9

Some Region become

- Eden
 - Survivor
 - Old
- } dynamically

Garbage First

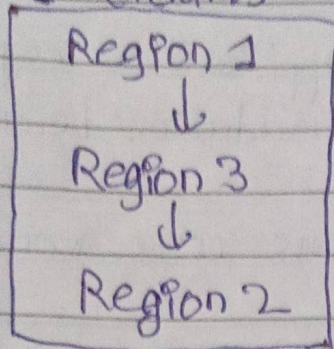
Suppose

Region 1 → 95% garbage

Region 2 → 20% garbage

Region 3 → 80% garbage

G1 cleans

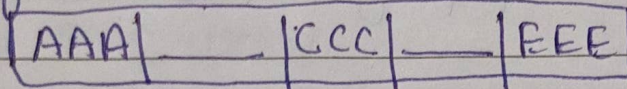


It always cleans regions containing the most garbage first. Hence the name Garbage First (G1).

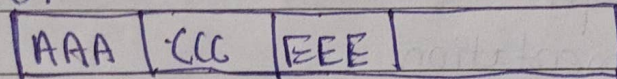
Compaction done Here, unlike CMS

G1 compact memory.

Before



After



Result: No fragmentation

Advantages:

- Low Pause time
- Parallel
- Concurrent
- Compaction
- Large heap support
- Predictable pause time